Cost-Aware Retail Search Inference Engine

Objective. Measure the trade-off between ranking quality and inference cost for four ranking strategies on the Amazon ESCI product-search dataset (US locale, Task 1 subset):

1. BM25 lexical baseline
2. Semantic ranking with sentence embeddings (`all-MiniLM-L6-v2`)
3. Cross-encoder reranking (`ms-marco-MiniLM-L-6-v2`)
4. Selective reranking, where only the top K semantic candidates are reranked by the cross-encoder

Selective reranking is a standard technique and is not claimed as novel. The contribution of this notebook is the implementation, the benchmark discipline, the uncertainty analysis and the documented cost trade-offs.

Scope. Every method ranks a *supplied candidate set*: the products that ESCI annotators judged for each query. The notebook does not retrieve candidates from a full catalog, so it is not an end-to-end retail search system. Section 5 states the consequences.

Structure. 1. Setup and data preparation, 2. Search index, 3. Baselines and inference optimization, 4. Reproducible evaluation, 5. Search service and operational benchmark.

Run conditions. CPU only. Run all cells from top to bottom in a fresh runtime. Timing values depend on the machine and the workload and are not latency guarantees.

1. Setup and Data Preparation

All fixed settings are defined in one cell. The only sources of randomness are the query sample and the bootstrap resampling, and both use seeded NumPy generators. Model inference runs in evaluation mode on CPU and is deterministic up to floating-point effects that depend on thread count and batch composition. Timing values are the only quantities expected to differ between runs.

```
%pip -q install rank-bm25 sentence-transformers
```

```
import hashlib
import json
import os
import platform
import shutil
import time
from pathlib import Path

import numpy as np
import pandas as pd
import requests
import torch
from rank_bm25 import BM25Okapi
from sentence_transformers import CrossEncoder, SentenceTransformer

# Reproducibility and workload settings
SEED = 42
N_EVAL_QUERIES = 200
K_VALUES = [5, 10, 20, 30]        # fixed selective-reranking depths; no further parameter search
N_BOOTSTRAP = 5000
ENCODE_BATCH_SIZE = 32
RERANK_BATCH_SIZE = 32

EMBEDDING_MODEL_ID = "sentence-transformers/all-MiniLM-L6-v2"
RERANKER_ID = "cross-encoder/ms-marco-MiniLM-L-6-v2"
RERANKER_MAX_LENGTH = 256
DEVICE = "cpu"

# ESCI labels mapped to graded relevance (Exact, Substitute, Complement, Irrelevant)
RELEVANCE = {"E": 3, "S": 2, "C": 1, "I": 0}

WORK_DIR = Path("/content/retail_search")
DATA_DIR = WORK_DIR / "data"
INDEX_DIR = WORK_DIR / "index"
RESULTS_DIR = WORK_DIR / "results"
for directory in (DATA_DIR, INDEX_DIR, RESULTS_DIR):
    directory.mkdir(parents=True, exist_ok=True)

print("Python:", platform.python_version())
print("PyTorch:", torch.__version__, "| CUDA available:", torch.cuda.is_available(),
      "| models run on:", DEVICE)
print("CPU count:", os.cpu_count(), "| torch intra-op threads:", torch.get_num_threads())
print("Disk free (GB):", round(shutil.disk_usage(WORK_DIR).free / 1e9, 2))
```

```
Python: 3.13.16
PyTorch: 2.11.0+cpu | CUDA available: False | models run on: cpu
CPU count: 2 | torch intra-op threads: 1
Disk free (GB): 93.57
```

```
BASE_URL = (
    "https://media.githubusercontent.com/media/"
    "amazon-science/esci-data/main/shopping_queries_dataset/"
)


def download_if_missing(file_name):
    """Download a dataset file once; reuse it when it is already on disk."""
    path = DATA_DIR / file_name
    if path.exists() and path.stat().st_size > 0:
        print(f"Reusing {path.name} ({path.stat().st_size / 1e6:.1f} MB)")
        return path
    partial = path.with_name(path.name + ".part")
    with requests.get(BASE_URL + file_name, stream=True, timeout=60) as response:
        response.raise_for_status()
        with open(partial, "wb") as handle:
            for chunk in response.iter_content(chunk_size=4 * 1024 * 1024):
                handle.write(chunk)
    partial.rename(path)
    print(f"Downloaded {path.name} ({path.stat().st_size / 1e6:.1f} MB)")
    return path


examples_path = download_if_missing("shopping_queries_dataset_examples.parquet")
products_path = download_if_missing("shopping_queries_dataset_products.parquet")
```

```
Downloaded shopping_queries_dataset_examples.parquet (51.3 MB)
Downloaded shopping_queries_dataset_products.parquet (1108.9 MB)
```

```
examples = pd.read_parquet(examples_path)
print("Examples shape:", examples.shape)

# Task 1 subset (small_version == 1), US locale
```

```
task1_us = examples[
    (examples["small_version"] == 1) & (examples["product_locale"] == "us")
].copy()
del examples
print("Task 1 US rows:", len(task1_us), "| unique queries:", task1_us["query_id"].nunique())
print(task1_us["split"].value_counts().to_string())

products = pd.read_parquet(
    products_path, columns=["product_id", "product_locale", "product_title"]
)
products = products[products["product_locale"] == "us"]

data = task1_us.merge(
    products[["product_id", "product_title"]],
    on="product_id",
    how="left",
    validate="many_to_one",
)
del products

data["product_title"] = data["product_title"].fillna("")
data["query"] = data["query"].fillna("")
data["relevance"] = data["esci_label"].map(RELEVANCE)

assert len(data) == len(task1_us), "The merge changed the number of judgment rows."
assert data["relevance"].notna().all(), "Unmapped ESCI labels found."
assert data["product_title"].ne("").all(), "Missing product titles after the merge."

print("Merged rows:", len(data))
print("Label distribution:", data["esci_label"].value_counts().to_dict())
```

```
Examples shape: (2621288, 9)
Task 1 US rows: 601354 | unique queries: 29844
split
train    419653
test     181701
Merged rows: 601354
Label distribution: {'E': 261527, 'S': 211191, 'I': 101447, 'C': 27189}
```

```
# Deterministic evaluation sample: whole queries, so each keeps its full candidate set.
eligible_test = data.loc[data["split"] == "test"].copy()

query_lookup = eligible_test[["query_id", "query"]].drop_duplicates()
assert query_lookup["query_id"].is_unique, "A query_id maps to multiple query strings."
assert len(query_lookup) >= N_EVAL_QUERIES, "Fewer eligible test queries than N_EVAL_QUERIES."

sampled_query_ids = (
    query_lookup["query_id"].sample(n=N_EVAL_QUERIES, random_state=SEED).tolist()
)

evaluation_sample = (
    eligible_test[eligible_test["query_id"].isin(sampled_query_ids)]
    .copy()
    .reset_index(drop=True)
)

assert evaluation_sample["query_id"].nunique() == N_EVAL_QUERIES
assert evaluation_sample["relevance"].notna().all()
assert evaluation_sample["product_title"].ne("").all()

candidates_per_query = evaluation_sample.groupby("query_id").size()
print("SEED:", SEED, "| eligible test queries:", len(query_lookup))
print("Sampled queries:", evaluation_sample["query_id"].nunique())
print("Candidate rows:", len(evaluation_sample))
print("Candidates per query: mean %.2f, min %d, max %d" % (
    candidates_per_query.mean(), candidates_per_query.min(), candidates_per_query.max()))
```

```
SEED: 42 | eligible test queries: 8956
Sampled queries: 200
Candidate rows: 4103
Candidates per query: mean 20.52, min 9, max 55
```

2. Search Index

Offline, one time: encode product titles and store the embedding matrix together with the title-to-row mapping. **Online, per query:** encode the query, look up the candidate rows in the index and compute dot products. The query path never encodes product titles, and a candidate title that is absent from the index is an error (Section 5).

The index is keyed by the exact title string, so products that share a title share one vector. It is built from the titles of the evaluation sample's candidate sets. A cached copy is reused only when the embedding model identifier, the complete ordered title list and a

checksum of the stored matrix all match, which avoids re-encoding in repeated runs within the same runtime.

```
embedder = SentenceTransformer(EMBEDDING_MODEL_ID, device=DEVICE)

# Unique titles in order of first appearance (deterministic given the sample)
index_titles = evaluation_sample["product_title"].drop_duplicates().tolist()

INDEX_VECTORS_PATH = INDEX_DIR / "product_embeddings.npy"
INDEX_META_PATH = INDEX_DIR / "index_meta.json"


def load_cached_index(titles):
    """Return (vectors, build_seconds) when a cache matches the model and titles, else None."""
    if not (INDEX_VECTORS_PATH.exists() and INDEX_META_PATH.exists()):
        return None
    meta = json.loads(INDEX_META_PATH.read_text())
    if meta["model_id"] != EMBEDDING_MODEL_ID or meta["titles"] != titles:
        return None
    vectors = np.load(INDEX_VECTORS_PATH)
    if vectors.dtype != np.float32 or vectors.shape != (len(titles), meta["dim"]):
        return None
    if meta.get("vectors_sha256") != hashlib.sha256(vectors.tobytes()).hexdigest():
        return None   # the stored matrix does not match what was written
    return vectors, meta["build_seconds"]


cached = load_cached_index(index_titles)
if cached is None:
    start = time.perf_counter()
    product_embeddings = embedder.encode(
        index_titles,
        batch_size=ENCODE_BATCH_SIZE,
        normalize_embeddings=True,
        convert_to_numpy=True,
        show_progress_bar=True,
    ).astype(np.float32, copy=False)
    index_build_seconds = time.perf_counter() - start
    np.save(INDEX_VECTORS_PATH, product_embeddings)
    INDEX_META_PATH.write_text(json.dumps({
        "model_id": EMBEDDING_MODEL_ID,
        "dim": int(product_embeddings.shape[1]),
        "build_seconds": index_build_seconds,
        "vectors_sha256": hashlib.sha256(product_embeddings.tobytes()).hexdigest(),
        "titles": index_titles,
    }))
    index_source = "built in this run"
else:
    product_embeddings, index_build_seconds = cached
    index_source = "loaded from cache"

title_to_idx = {title: row for row, title in enumerate(index_titles)}

print("Index:", index_source)
print("Offline index preparation: %.2f seconds (one-time, excluded from query-time costs)"
      % index_build_seconds)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
modules.json: 100%  349/349 [00:00<00:00, 10.2kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 4.77kB/s]
README.md: 100%  10.5k/10.5k [00:00<00:00, 294kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 1.26kB/s]
config.json: 100%  612/612 [00:00<00:00, 43.1kB/s]
model.safetensors: reconstructing file: 100%  90.9MB / 90.9MB, 8.81MB/s
model.safetensors: downloading bytes:  85.0MB, 8.08MB/s
Loading weights: 100%  103/103 [00:00<00:00, 1339.56it/s]
tokenizer_config.json: 100%  350/350 [00:00<00:00, 6.32kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 4.93MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 6.58MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 1.32kB/s]
config.json: 100%  190/190 [00:00<00:00, 3.19kB/s]
Batches: 100%  128/128 [01:10<00:00,  6.28it/s]
Index: built in this run
Offline index preparation: 70.93 seconds (one-time, excluded from query-time costs)
```

```python
# Index consistency: vectors and title mapping must stay aligned.
assert len(title_to_idx) == len(index_titles), "Duplicate titles in the index."
assert product_embeddings.shape[0] == len(title_to_idx)
assert product_embeddings.dtype == np.float32
assert sorted(title_to_idx.values()) == list(range(len(index_titles)))
assert np.allclose(np.linalg.norm(product_embeddings, axis=1), 1.0, atol=1e-3), \
    "Embeddings are not L2-normalised."

missing_titles = set(evaluation_sample["product_title"]) - set(title_to_idx)
assert not missing_titles, f"{len(missing_titles)} evaluation titles are not indexed."

# Alignment spot check: re-encode a seeded random subset and compare with the stored rows.
# With 16 probes this detects systematic misalignment (offsets, reordering, a stale matrix).
# It is not designed to detect an isolated swap of two rows; the checksum above guards the cache.
probe_rows = np.random.default_rng(SEED).choice(
    len(index_titles), size=min(16, len(index_titles)), replace=False
)
re_encoded = embedder.encode(
    [index_titles[row] for row in probe_rows],
    batch_size=ENCODE_BATCH_SIZE,
    normalize_embeddings=True,
    convert_to_numpy=True,
    show_progress_bar=False,
)
assert re_encoded.shape[1] == product_embeddings.shape[1]
assert np.allclose(re_encoded, product_embeddings[probe_rows], atol=1e-4), \
    "Stored vectors do not match their titles."

print("Embedding matrix:", product_embeddings.shape, product_embeddings.dtype)
print("Indexed titles:", len(title_to_idx))
print("Index consistency checks passed.")
```

```
Embedding matrix: (4073, 384) float32
Indexed titles: 4073
Index consistency checks passed.
```

3. Baselines and Inference Optimization

This section produces the **orderings** and the **cost measurements** for every method. Quality metrics for all orderings are computed in Section 4 by a single evaluation implementation, so the notebook contains exactly one NDCG code path.

Conventions used here:

- Orderings are arrays of local candidate positions within each query. All of them come from stable sorts, so tied scores keep the dataset row order.
- BM25 is fitted separately for each query on that query's candidate titles only. Its inverse document frequencies therefore come from roughly 20 documents and are noisy. This is a weak lexical baseline, not a tuned one.
- Cross-encoder cost is measured as the wall-clock time of the rerank step on CPU with batch size 32, after one warm-up call. Pair counts are exact.

```
assert evaluation_sample.index.equals(pd.RangeIndex(len(evaluation_sample)))

QUERY_IDS = evaluation_sample["query_id"].drop_duplicates().tolist()
query_rows = evaluation_sample.groupby("query_id", sort=False).indices   # query_id -> row positions

titles_all = evaluation_sample["product_title"].to_numpy()
labels_all = evaluation_sample["esci_label"].to_numpy()
relevance_all = evaluation_sample["relevance"].to_numpy()
query_text = {
    query_id: evaluation_sample["query"].iat[query_rows[query_id][0]] for query_id in QUERY_IDS
}

BM25_NAME = "BM25"
SEMANTIC_NAME = "Semantic only"
CROSS_ENCODER_NAME = "Cross-encoder, all candidates (reference)"


def selective_name(k):
    return f"Selective rerank K={k}"


orderings = {}   # method -> {query_id: local candidate positions, best first}
cost = {}        # method -> cross-encoder pair count and seconds
```

```
def tokenize(text):
    return str(text).lower().split()


bm25_order = {}
start = time.perf_counter()
for query_id in QUERY_IDS:
    rows = query_rows[query_id]
    bm25 = BM25Okapi([tokenize(title) for title in titles_all[rows]])
    scores = np.asarray(bm25.get_scores(tokenize(query_text[query_id])))
    bm25_order[query_id] = np.argsort(-scores, kind="stable")
bm25_seconds = time.perf_counter() - start

orderings[BM25_NAME] = bm25_order
cost[BM25_NAME] = {"pairs": 0, "seconds": 0.0}
print("BM25 (per-query fit and scoring, all %d queries): %.2f s | %.2f ms/query"
      % (len(QUERY_IDS), bm25_seconds, bm25_seconds / len(QUERY_IDS) * 1000))
```

```
BM25 (per-query fit and scoring, all 200 queries): 0.08 s | 0.41 ms/query
```

```
# Query embeddings are encoded in one batched call for the offline evaluation.
start = time.perf_counter()
query_embeddings = embedder.encode(
    [query_text[query_id] for query_id in QUERY_IDS],
    batch_size=ENCODE_BATCH_SIZE,
    normalize_embeddings=True,
    convert_to_numpy=True,
    show_progress_bar=False,
)
query_encoding_seconds = time.perf_counter() - start

semantic_scores = {}
semantic_order = {}
start = time.perf_counter()
for position, query_id in enumerate(QUERY_IDS):
    rows = query_rows[query_id]
    index_rows = [title_to_idx[title] for title in titles_all[rows]]
    scores = product_embeddings[index_rows] @ query_embeddings[position]
    semantic_scores[query_id] = scores
    semantic_order[query_id] = np.argsort(-scores, kind="stable")
semantic_scoring_seconds = time.perf_counter() - start

orderings[SEMANTIC_NAME] = semantic_order
cost[SEMANTIC_NAME] = {"pairs": 0, "seconds": 0.0}
print("Semantic, batched query encoding (%d queries): %.3f s | %.2f ms/query"
      % (len(QUERY_IDS), query_encoding_seconds, query_encoding_seconds / len(QUERY_IDS) * 1000))
print("Semantic, candidate scoring and sorting: %.3f s | %.2f ms/query"
      % (semantic_scoring_seconds, semantic_scoring_seconds / len(QUERY_IDS) * 1000))
```

```
Semantic, batched query encoding (200 queries): 0.775 s | 3.87 ms/query
Semantic, candidate scoring and sorting: 0.006 s | 0.03 ms/query
```

```
# Query-encoding batching experiment: one call per query versus one batched call.
# Both paths request normalised embeddings, so the vectors are directly comparable.
texts = [query_text[query_id] for query_id in QUERY_IDS]
embedder.encode(texts[:4], normalize_embeddings=True, show_progress_bar=False)   # warm-up
```

```

start = time.perf_counter()
individual = np.vstack([
    embedder.encode([text], normalize_embeddings=True, convert_to_numpy=True,
                    show_progress_bar=False)[0]
    for text in texts
])
individual_seconds = time.perf_counter() - start

start = time.perf_counter()
batched = embedder.encode(
    texts, batch_size=ENCODE_BATCH_SIZE, normalize_embeddings=True,
    convert_to_numpy=True, show_progress_bar=False,
)
batched_seconds = time.perf_counter() - start

max_difference = float(np.max(np.abs(individual - batched)))
assert max_difference < 1e-4, "Batching changed the query embeddings."

print("Individual: %.3f s (%.2f ms/query)" % (individual_seconds, individual_seconds / len(texts) * 1000))
print("Batched (batch_size=%d): %.3f s (%.2f ms/query)"
      % (ENCODE_BATCH_SIZE, batched_seconds, batched_seconds / len(texts) * 1000))
print("Speed-up: %.1fx | maximum embedding difference: %.2e" % (
    individual_seconds / batched_seconds, max_difference))
print("Note: batching applies to offline evaluation. The online service in Section 5 "
      "receives one query per call and therefore encodes with a batch of one.")
```

```
Individual: 3.027 s (15.14 ms/query)
Batched (batch_size=32): 0.771 s (3.85 ms/query)
Speed-up: 3.9x | maximum embedding difference: 1.24e-07
Note: batching applies to offline evaluation. The online service in Section 5 receives one query per call and therefore encc
```

```

reranker = CrossEncoder(RERANKER_ID, device=DEVICE, max_length=RERANKER_MAX_LENGTH)


def rerank_top_k(query, titles, order, k):
    """Selective reranking policy, shared by the evaluation and the service.

    order  : candidate positions sorted by semantic score, best first
    titles : candidate titles indexed by local position
    k      : number of top semantic candidates to rerank (None reranks all)

    Returns (final_order, cross_scores). The first len(cross_scores) entries of
    final_order are the reranked candidates, sorted by cross-encoder score, and
    cross_scores[i] belongs to final_order[i]. The remaining candidates follow in
    their semantic order. Semantic similarities and cross-encoder logits are never
    compared with each other.
    """
    depth = len(order) if k is None else min(k, len(order))
    if depth == 0:
        return order, np.empty(0, dtype=np.float32)
    head = order[:depth]
    scores = np.asarray(
        reranker.predict(
            [(query, titles[position]) for position in head],
            batch_size=RERANK_BATCH_SIZE,
            show_progress_bar=False,
        ),
        dtype=np.float32,
    )
    by_score = np.argsort(-scores, kind="stable")
    return np.concatenate([head[by_score], order[depth:]]), scores[by_score]


# Warm-up call so first-call overhead does not distort the timings below.
first_rows = query_rows[QUERY_IDS[0]]
reranker.predict([(query_text[QUERY_IDS[0]], titles_all[first_rows[0]])], show_progress_bar=False)

rerank_depths = {selective_name(k): k for k in K_VALUES}
rerank_depths[CROSS_ENCODER_NAME] = None   # reference: every candidate is scored

for method, depth in rerank_depths.items():
    final_orders, pairs_scored = {}, 0
    start = time.perf_counter()
    for query_id in QUERY_IDS:
        rows = query_rows[query_id]
        final_orders[query_id], cross_scores = rerank_top_k(
            query_text[query_id], titles_all[rows], semantic_order[query_id], depth
        )
        pairs_scored += len(cross_scores)
    seconds = time.perf_counter() - start
```

```
    orderings[method] = final_orders
    cost[method] = {"pairs": pairs_scored, "seconds": seconds}
    print(f"{method}: {pairs_scored} pairs, {seconds:.2f} s")
```

```
config.json: 100%  794/794 [00:00<00:00, 30.5kB/s]
model.safetensors: reconstructing file: 100%  90.9MB / 90.9MB, 8.90MB/s
model.safetensors: downloading bytes:  86.0MB, 8.30MB/s
Loading weights: 100%  105/105 [00:00<00:00, 2808.47it/s]
tokenizer_config.json: 100%  1.33k/1.33k [00:00<00:00, 54.9kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 9.09MB/s]
tokenizer.json: 100%  711k/711k [00:00<00:00, 23.1MB/s]
special_tokens_map.json: 100%  132/132 [00:00<00:00, 7.38kB/s]
Selective rerank K=5: 1000 pairs, 22.63 s
Selective rerank K=10: 1999 pairs, 40.43 s
Selective rerank K=20: 3321 pairs, 75.59 s
Selective rerank K=30: 3725 pairs, 79.14 s
Cross-encoder, all candidates (reference): 4103 pairs, 107.75 s
```

```
total_candidate_pairs = len(evaluation_sample)
cost_table = pd.DataFrame([
    {
        "Method": method,
        "Cross-encoder pairs": values["pairs"],
        "Pairs scored (% of candidates)": 100 * values["pairs"] / total_candidate_pairs,
        "Cross-encoder seconds": values["seconds"],
        "Cross-encoder ms/query": 1000 * values["seconds"] / len(QUERY_IDS),
    }
    for method, values in cost.items()
])
display(cost_table.round(2))
print("Conditions: CPU, rerank batch size %d, %d queries, %d candidate pairs in total, "
      "one warm-up call, sequential per-query calls." % (
          RERANK_BATCH_SIZE, len(QUERY_IDS), total_candidate_pairs))
print("Non-cross-encoder costs: BM25 %.2f s in total, semantic query encoding %.3f s (batched), "
      "semantic scoring %.3f s." % (bm25_seconds, query_encoding_seconds, semantic_scoring_seconds))
```

	Method	Cross-encoder pairs	Pairs scored (% of candidates)	Cross-encoder seconds	Cross-encoder ms/query
0	BM25	0	0.00	0.00	0.00
1	Semantic only	0	0.00	0.00	0.00
2	Selective rerank K=5	1000	24.37	22.63	113.17
3	Selective rerank K=10	1999	48.72	40.43	202.14
4	Selective rerank K=20	3321	80.94	75.59	377.96
5	Selective rerank K=30	3725	90.79	79.14	395.71
6	Cross-encoder, all candidates (reference)	4103	100.00	107.75	538.76

4. Reproducible Evaluation

Metric definitions

- **NDCG@10** uses the relevance value itself as the gain (E = 3, S = 2, C = 1, I = 0) and a log2 rank discount. The ideal ranking is computed from the same candidate set. A query without any positive relevance scores 0. This matches the linear-gain definition of `sklearn.metrics.ndcg_score` when scores are untied, which the tests below confirm.
- **Tie handling.** Every method is evaluated from an explicit ordering produced by a stable sort. Tied scores keep the dataset row order. Score-averaging over ties is not used.
- **Exact in top k.** The share of queries that have at least one candidate labelled `E` (Exact) among the first k results. The ceiling row reports the share of queries that have any `E` candidate at all, which bounds this metric from above.
- **Paired comparisons.** All methods are evaluated on the same 200 queries. Differences are computed per query and a 95% percentile bootstrap interval is obtained by resampling queries (not rows) with replacement.

```
def ndcg_at_k(ranked_relevance, k=10):
    """NDCG@k for relevance values listed in ranked order (linear gain, log2 discount)."""
    ranked_relevance = np.asarray(ranked_relevance, dtype=float)
    discounts = 1.0 / np.log2(np.arange(2, k + 2))
    top = ranked_relevance[:k]
```

```
    dcg = float(np.sum(top * discounts[: len(top)]))
    ideal = np.sort(ranked_relevance)[::-1][:k]
    ideal_dcg = float(np.sum(ideal * discounts[: len(ideal)]))
    return dcg / ideal_dcg if ideal_dcg > 0 else 0.0


def exact_in_top_k(ranked_labels, k):
    """True when at least one of the first k results is labelled E (Exact)."""
    return bool(np.any(np.asarray(ranked_labels)[:k] == "E"))


def evaluate_method(method_orderings):
    """Per-query metrics for one method, indexed by query_id in QUERY_IDS order."""
    records = []
    for query_id in QUERY_IDS:
        rows = query_rows[query_id]
        order = method_orderings[query_id]
        assert np.array_equal(np.sort(order), np.arange(len(rows))), \
            "An ordering must be a permutation of the query's candidates."
        records.append({
            "query_id": query_id,
            "ndcg_at_10": ndcg_at_k(relevance_all[rows][order], 10),
            "exact_top5": exact_in_top_k(labels_all[rows][order], 5),
            "exact_top10": exact_in_top_k(labels_all[rows][order], 10),
        })
    return pd.DataFrame(records).set_index("query_id")


# Metric self-checks
assert np.isclose(ndcg_at_k([3, 2, 1, 0]), 1.0)
assert ndcg_at_k([0, 0, 0]) == 0.0
assert np.isclose(ndcg_at_k([0, 3]), 3 / np.log2(3) / 3)

from sklearn.metrics import ndcg_score  # used only to cross-check the implementation

check_rng = np.random.default_rng(SEED)
for _ in range(200):
    n_items = int(check_rng.integers(3, 40))
    relevance = check_rng.integers(0, 4, size=n_items)
    scores = check_rng.random(n_items)                  # continuous scores, so no ties
    order = np.argsort(-scores, kind="stable")
    assert np.isclose(ndcg_at_k(relevance[order], 10), ndcg_score([relevance], [scores], k=10))
print("NDCG@10 implementation agrees with sklearn on 200 synthetic untied cases.")
```

```
NDCG@10 implementation agrees with sklearn on 200 synthetic untied cases.
```

```
method_names = (
    [BM25_NAME, SEMANTIC_NAME]
    + [selective_name(k) for k in K_VALUES]
    + [CROSS_ENCODER_NAME]
)

per_query = {method: evaluate_method(orderings[method]) for method in method_names}

benchmark = pd.DataFrame([
    {
        "Method": method,
        "NDCG@10": per_query[method]["ndcg_at_10"].mean(),
        "Exact in top 5": per_query[method]["exact_top5"].mean(),
        "Exact in top 10": per_query[method]["exact_top10"].mean(),
        "Cross-encoder pairs": cost[method]["pairs"],
        "Pairs scored (%)": 100 * cost[method]["pairs"] / total_candidate_pairs,
        "Cross-encoder seconds": cost[method]["seconds"],
    }
    for method in method_names
])

queries_with_exact = float(np.mean([
    np.any(labels_all[query_rows[query_id]] == "E") for query_id in QUERY_IDS
]))

display(benchmark.round(4))
print("Ceiling for the Exact-in-top-k metrics: %.3f of queries contain at least one E candidate."
      % queries_with_exact)
print("Sample: %d queries, %d candidate rows, SEED=%d. CPU timings are machine dependent."
      % (len(QUERY_IDS), total_candidate_pairs, SEED))
benchmark.to_csv(RESULTS_DIR / "benchmark_table.csv", index=False)
```

	Method	NDCG@10	Exact in top 5	Exact in top 10	Cross-encoder pairs	Pairs scored (%)	Cross-encoder seconds
0	BM25	0.8178	0.905	0.97	0	0.0000	0.0000
1	Semantic only	0.8445	0.940	0.99	0	0.0000	0.0000
2	Selective rerank K=5	0.8457	0.940	0.99	1000	24.3724	22.6348
3	Selective rerank K=10	0.8428	0.920	0.99	1999	48.7204	40.4288
4	Selective rerank K=20	0.8474	0.915	0.99	3321	80.9408	75.5917
5	Selective rerank K=30	0.8471	0.915	0.99	3725	90.7872	79.1423
6	Cross-encoder, all candidates (reference)	0.8465	0.915	0.99	4103	100.0000	107.7518

```python
def paired_bootstrap_ci(differences, rng, n_bootstrap=N_BOOTSTRAP):
    """95% percentile interval for the mean of paired per-query differences."""
    n_queries = len(differences)
    means = np.empty(n_bootstrap)
    for i in range(n_bootstrap):
        means[i] = differences[rng.integers(0, n_queries, size=n_queries)].mean()
    return float(np.percentile(means, 2.5)), float(np.percentile(means, 97.5))


def per_query_ndcg(method):
    return per_query[method]["ndcg_at_10"].to_numpy()


# The first five comparisons keep the order of the earlier analysis; the last two are appended.
comparisons = [
    ("K=5 minus semantic", selective_name(5), SEMANTIC_NAME),
    ("K=20 minus semantic", selective_name(20), SEMANTIC_NAME),
    ("K=30 minus semantic", selective_name(30), SEMANTIC_NAME),
    ("K=20 minus K=5", selective_name(20), selective_name(5)),
    ("K=30 minus K=20", selective_name(30), selective_name(20)),
    ("Semantic minus BM25", SEMANTIC_NAME, BM25_NAME),
    ("Cross-encoder (all) minus semantic", CROSS_ENCODER_NAME, SEMANTIC_NAME),
]

bootstrap_rng = np.random.default_rng(SEED)
ci_rows = []
for label, method_a, method_b in comparisons:
    differences = per_query_ndcg(method_a) - per_query_ndcg(method_b)
    lower, upper = paired_bootstrap_ci(differences, bootstrap_rng)
    ci_rows.append({
        "Comparison": label,
        "Mean NDCG difference": differences.mean(),
        "95% CI lower": lower,
        "95% CI upper": upper,
        "CI includes zero": bool(lower <= 0.0 <= upper),
    })

ci_table = pd.DataFrame(ci_rows)
display(ci_table.round(5))
print("Paired bootstrap over queries: %d resamples, SEED=%d, %d queries." % (
    N_BOOTSTRAP, SEED, len(QUERY_IDS)))
ci_table.to_csv(RESULTS_DIR / "bootstrap_ci.csv", index=False)
```

	Comparison	Mean NDCG difference	95% CI lower	95% CI upper	CI includes zero
0	K=5 minus semantic	0.00121	-0.00455	0.00688	True
1	K=20 minus semantic	0.00298	-0.00948	0.01562	True
2	K=30 minus semantic	0.00268	-0.00955	0.01555	True
3	K=20 minus K=5	0.00177	-0.00919	0.01304	True
4	K=30 minus K=20	-0.00030	-0.00276	0.00184	True
5	Semantic minus BM25	0.02670	0.00907	0.04450	False
6	Cross-encoder (all) minus semantic	0.00203	-0.01064	0.01529	True

```
Paired bootstrap over queries: 5000 resamples, SEED=42, 200 queries.
```

```python
# Interpretation generated from the computed intervals, so it cannot drift from the results.
for _, row in ci_table.iterrows():
    if row["CI includes zero"]:
        verdict = ("the 95% interval includes zero, so the observed difference is not "
                   "statistically supported on this sample")
    else:
```

```
        verdict = "the 95% interval excludes zero, so a difference is supported on this sample"
    print("%s: mean %+.5f, %s." % (row["Comparison"], row["Mean NDCG difference"], verdict))

print("\nCaveat: seven intervals are reported without a multiple-comparison correction.")
```

```
K=5 minus semantic: mean +0.00121, the 95% interval includes zero, so the observed difference is not statistically supported
K=20 minus semantic: mean +0.00298, the 95% interval includes zero, so the observed difference is not statistically supporte
K=30 minus semantic: mean +0.00268, the 95% interval includes zero, so the observed difference is not statistically supporte
K=20 minus K=5: mean +0.00177, the 95% interval includes zero, so the observed difference is not statistically supported on
K=30 minus K=20: mean -0.00030, the 95% interval includes zero, so the observed difference is not statistically supported or
Semantic minus BM25: mean +0.02670, the 95% interval excludes zero, so a difference is supported on this sample.
Cross-encoder (all) minus semantic: mean +0.00203, the 95% interval includes zero, so the observed difference is not statist

Caveat: seven intervals are reported without a multiple-comparison correction.
```

What the evaluation establishes. The table gives the quality of each ranking policy on one fixed sample of 200 queries together with the exact number of cross-encoder pairs scored and the measured CPU time. The intervals state which differences in mean NDCG@10 the sample can distinguish from zero. An observed difference whose interval includes zero is reported as an observation only, and no improvement is claimed for it.

What remains uncertain.

- One sample of 200 queries with about 20 candidates each. Larger or differently distributed samples may change the intervals.
- Candidate sets are the judged ESCI products for each query, so the benchmark measures reranking of a small judged pool, not retrieval from a catalog.
- Pair counts and the quality metrics are exact for this sample. Seconds are machine and workload dependent and describe sequential CPU inference with batch size 32.
- The cross-encoder was trained on MS MARCO and was not adapted to ESCI.
- Results from earlier exploratory runs used a different query sample (the first 200 test queries, about 29 candidates per query). They are not comparable with the numbers above and were removed.

```
# Reconciliation with the values recorded in the earlier version of this notebook
# (same SEED and N_EVAL_QUERIES). The table and intervals are recomputed above; this cell
# only prints the differences. Timing is machine dependent and is not compared.
REFERENCE_RUN = {
    "candidate_rows": 4103,
    "mean_candidates": 20.52,
    "min_candidates": 9,
    "max_candidates": 55,
    "ndcg": {SEMANTIC_NAME: 0.8445, selective_name(5): 0.8457, selective_name(10): 0.8428,
             selective_name(20): 0.8474, selective_name(30): 0.8471},
    "exact_top5": {SEMANTIC_NAME: 0.940, selective_name(5): 0.940, selective_name(10): 0.920,
                   selective_name(20): 0.915, selective_name(30): 0.915},
    "exact_top10": {SEMANTIC_NAME: 0.99, selective_name(5): 0.99, selective_name(10): 0.99,
                    selective_name(20): 0.99, selective_name(30): 0.99},
    "pairs": {SEMANTIC_NAME: 0, selective_name(5): 1000, selective_name(10): 1999,
              selective_name(20): 3321, selective_name(30): 3725},
    "ci": {"K=5 minus semantic": (0.00121, -0.00455, 0.00688),
           "K=20 minus semantic": (0.00298, -0.00948, 0.01562),
           "K=30 minus semantic": (0.00268, -0.00955, 0.01555),
           "K=20 minus K=5": (0.00177, -0.00919, 0.01304),
           "K=30 minus K=20": (-0.00030, -0.00276, 0.00184)},
}

rows = [
    ("candidate rows", REFERENCE_RUN["candidate_rows"], total_candidate_pairs, 0),
    ("mean candidates per query", REFERENCE_RUN["mean_candidates"], candidates_per_query.mean(), 0.005),
    ("min candidates per query", REFERENCE_RUN["min_candidates"], candidates_per_query.min(), 0),
    ("max candidates per query", REFERENCE_RUN["max_candidates"], candidates_per_query.max(), 0),
]
bench = benchmark.set_index("Method")
for method, value in REFERENCE_RUN["ndcg"].items():
    rows.append((f"NDCG@10, {method}", value, bench.loc[method, "NDCG@10"], 5e-4))
for method, value in REFERENCE_RUN["exact_top5"].items():
    rows.append((f"Exact top 5, {method}", value, bench.loc[method, "Exact in top 5"], 1e-9))
for method, value in REFERENCE_RUN["exact_top10"].items():
    rows.append((f"Exact top 10, {method}", value, bench.loc[method, "Exact in top 10"], 1e-9))
for method, value in REFERENCE_RUN["pairs"].items():
    rows.append((f"Pairs, {method}", value, bench.loc[method, "Cross-encoder pairs"], 0))
ci_lookup = ci_table.set_index("Comparison")
for label, (mean, lower, upper) in REFERENCE_RUN["ci"].items():
    rows.append((f"CI mean, {label}", mean, ci_lookup.loc[label, "Mean NDCG difference"], 5e-4))
    rows.append((f"CI lower, {label}", lower, ci_lookup.loc[label, "95% CI lower"], 5e-4))
    rows.append((f"CI upper, {label}", upper, ci_lookup.loc[label, "95% CI upper"], 5e-4))

reconciliation = pd.DataFrame(rows, columns=["Quantity", "Earlier run", "This run", "Tolerance"])
reconciliation["Difference"] = reconciliation["This run"] - reconciliation["Earlier run"]
reconciliation["Status"] = np.where(
```

```
    reconciliation["Difference"].abs() <= reconciliation["Tolerance"] + 1e-12, "match", "DIFFERS"
)
display(reconciliation.round(5))
print("Items that differ:", int((reconciliation["Status"] == "DIFFERS").sum()),
      "of", len(reconciliation))
print("Indexed titles in this run: %d. The earlier run's index held 9,298 titles because it also "
      "contained titles from an earlier sample, so a smaller count is expected." % len(title_to_idx))
```

	Quantity	Earlier run	This run	Tolerance	Difference	Status
0	candidate rows	4103.00000	4103.00000	0.0000	0.00000	match
1	mean candidates per query	20.52000	20.51500	0.0050	-0.00500	match
2	min candidates per query	9.00000	9.00000	0.0000	0.00000	match
3	max candidates per query	55.00000	55.00000	0.0000	0.00000	match
4	NDCG@10, Semantic only	0.84450	0.84446	0.0005	-0.00004	match
5	NDCG@10, Selective rerank K=5	0.84570	0.84567	0.0005	-0.00003	match
6	NDCG@10, Selective rerank K=10	0.84280	0.84285	0.0005	0.00005	match
7	NDCG@10, Selective rerank K=20	0.84740	0.84744	0.0005	0.00004	match
8	NDCG@10, Selective rerank K=30	0.84710	0.84714	0.0005	0.00004	match
9	Exact top 5, Semantic only	0.94000	0.94000	0.0000	0.00000	match
10	Exact top 5, Selective rerank K=5	0.94000	0.94000	0.0000	0.00000	match
11	Exact top 5, Selective rerank K=10	0.92000	0.92000	0.0000	0.00000	match
12	Exact top 5, Selective rerank K=20	0.91500	0.91500	0.0000	0.00000	match
13	Exact top 5, Selective rerank K=30	0.91500	0.91500	0.0000	0.00000	match
14	Exact top 10, Semantic only	0.99000	0.99000	0.0000	0.00000	match
15	Exact top 10, Selective rerank K=5	0.99000	0.99000	0.0000	0.00000	match
16	Exact top 10, Selective rerank K=10	0.99000	0.99000	0.0000	0.00000	match
17	Exact top 10, Selective rerank K=20	0.99000	0.99000	0.0000	0.00000	match
18	Exact top 10, Selective rerank K=30	0.99000	0.99000	0.0000	0.00000	match
19	Pairs, Semantic only	0.00000	0.00000	0.0000	0.00000	match
20	Pairs, Selective rerank K=5	1000.00000	1000.00000	0.0000	0.00000	match
21	Pairs, Selective rerank K=10	1999.00000	1999.00000	0.0000	0.00000	match
22	Pairs, Selective rerank K=20	3321.00000	3321.00000	0.0000	0.00000	match
23	Pairs, Selective rerank K=30	3725.00000	3725.00000	0.0000	0.00000	match
24	CI mean, K=5 minus semantic	0.00121	0.00121	0.0005	0.00000	match
25	CI lower, K=5 minus semantic	-0.00455	-0.00455	0.0005	0.00000	match
26	CI upper, K=5 minus semantic	0.00688	0.00688	0.0005	0.00000	match
27	CI mean, K=20 minus semantic	0.00298	0.00298	0.0005	-0.00000	match
28	CI lower, K=20 minus semantic	-0.00948	-0.00948	0.0005	-0.00000	match
29	CI upper, K=20 minus semantic	0.01562	0.01562	0.0005	0.00000	match
30	CI mean, K=30 minus semantic	0.00268	0.00268	0.0005	0.00000	match
31	CI lower, K=30 minus semantic	-0.00955	-0.00955	0.0005	-0.00000	match
32	CI upper, K=30 minus semantic	0.01555	0.01555	0.0005	-0.00000	match
33	CI mean, K=20 minus K=5	0.00177	0.00177	0.0005	-0.00000	match
34	CI lower, K=20 minus K=5	-0.00919	-0.00919	0.0005	-0.00000	match
35	CI upper, K=20 minus K=5	0.01304	0.01304	0.0005	-0.00000	match
36	CI mean, K=30 minus K=20	-0.00030	-0.00030	0.0005	0.00000	match
37	CI lower, K=30 minus K=20	-0.00276	-0.00276	0.0005	-0.00000	match
38	CI upper, K=30 minus K=20	0.00184	0.00184	0.0005	0.00000	match

```
Items that differ: 0 of 39
Indexed titles in this run: 4073. The earlier run's index held 9,298 titles because it also contained titles from an earli
```

5. Search Service and Operational Benchmark

`search_candidates()` **ranks a supplied candidate set** for one query. It does not retrieve candidates from the full catalog. The caller must provide the candidate rows, and every candidate title must already be in the index built in Section 2. The function never encodes product titles, never modifies the index and raises an error for an unindexed title.

Modes: `rerank_k=0` gives semantic-only ranking. `rerank_k=K` reranks the top K semantic candidates with the cross-encoder and keeps the remaining candidates in their semantic order after them. The ranking policy is the `rerank_top_k` function from Section 3, so the service and the evaluation share one implementation.

Return value: a tuple `(results, stats)`. `results` holds the original candidate columns plus `semantic_score`, `cross_score` (NaN for candidates that were not reranked) and `final_rank`, limited to `top_n` rows. Scores from the two models are on different scales and are not comparable with each other.

```
RESERVED_OUTPUT_COLUMNS = ("semantic_score", "cross_score", "final_rank")


def _require_int(value, name, minimum):
    if isinstance(value, (bool, np.bool_)) or not isinstance(value, (int, np.integer)) or value < minimum:
        raise ValueError(f"{name} must be an integer >= {minimum}.")


def search_candidates(query, candidates, top_n=10, rerank_k=0):
    """Rank a supplied candidate set for one query.

    Parameters
    ----------
    query      : non-empty string
    candidates : DataFrame with a product_title column; every title must be in the index
    top_n      : number of rows to return (>= 1)
    rerank_k   : 0 for semantic-only ranking, otherwise the number of top semantic
                 candidates to rerank with the cross-encoder (clamped to the candidate count)

    Returns
    -------
    (results, stats) where stats contains candidate_count, reranked_count,
    query_encoding_ms, semantic_scoring_ms and rerank_ms.
    """
    if not isinstance(query, str) or not query.strip():
        raise ValueError("query must be a non-empty string.")
    if not isinstance(candidates, pd.DataFrame) or "product_title" not in candidates.columns:
        raise ValueError("candidates must be a DataFrame with a product_title column.")
    _require_int(top_n, "top_n", 1)
    _require_int(rerank_k, "rerank_k", 0)
    clashing = [name for name in RESERVED_OUTPUT_COLUMNS if name in candidates.columns]
    if clashing:
        raise ValueError(f"candidates already contains reserved columns: {clashing}.")

    stats = {"candidate_count": len(candidates), "reranked_count": 0,
             "query_encoding_ms": 0.0, "semantic_scoring_ms": 0.0, "rerank_ms": 0.0}
    if candidates.empty:
        return pd.DataFrame(columns=list(candidates.columns) + list(RESERVED_OUTPUT_COLUMNS)), stats

    titles = candidates["product_title"]
    if titles.isna().any() or not titles.map(lambda t: isinstance(t, str) and t.strip() != "").all():
        raise ValueError("candidates contain null or empty product titles.")
    titles = titles.to_numpy()
    unindexed = {title for title in titles if title not in title_to_idx}
    if unindexed:
        raise ValueError(
            f"{len(unindexed)} candidate titles are not in the index. Extend the index offline "
            "(Section 2); the query path does not encode product titles."
        )

    start = time.perf_counter()
    query_embedding = embedder.encode(
        [query], normalize_embeddings=True, convert_to_numpy=True, show_progress_bar=False
    )[0]
    stats["query_encoding_ms"] = (time.perf_counter() - start) * 1000

    start = time.perf_counter()
    index_rows = [title_to_idx[title] for title in titles]
    semantic = product_embeddings[index_rows] @ query_embedding
    order = np.argsort(-semantic, kind="stable")
    stats["semantic_scoring_ms"] = (time.perf_counter() - start) * 1000

    start = time.perf_counter()
    final_order, cross_scores = rerank_top_k(query, titles, order, rerank_k)
    stats["rerank_ms"] = (time.perf_counter() - start) * 1000
    stats["reranked_count"] = len(cross_scores)

```

```
    results = candidates.iloc[final_order].reset_index(drop=True)
    results["semantic_score"] = semantic[final_order]
    cross_column = np.full(len(results), np.nan, dtype=np.float32)
    cross_column[: len(cross_scores)] = cross_scores
    results["cross_score"] = cross_column
    results["final_rank"] = np.arange(1, len(results) + 1)

    for name in ("query_encoding_ms", "semantic_scoring_ms", "rerank_ms"):
        stats[name] = round(stats[name], 3)
    return results.head(top_n), stats


# Demonstration on one evaluation query
demo_query_id = QUERY_IDS[0]
demo_frame = evaluation_sample.iloc[query_rows[demo_query_id]].reset_index(drop=True)
demo_results, demo_stats = search_candidates(
    query_text[demo_query_id], demo_frame, top_n=5, rerank_k=5
)
display(demo_results[["final_rank", "product_title", "esci_label", "semantic_score", "cross_score"]])
print(demo_stats)
```

	final_rank	product_title	esci_label	semantic_score	cross_score
0	1	Aftermarket Replacement Driver and Passenger S...	I	0.699044	5.345150
1	2	DWVO Compatible with 2002 2003 2004 2005 2006 ...	S	0.553432	2.620811
2	3	HEADLIGHTSDEPOT Black Housing Halogen Headligh...	E	0.560579	-1.746852
3	4	HEADLIGHTSDEPOT Halogen Headlight Compatible W...	E	0.583948	-1.777377
4	5	Auto Ventshade AVS 37430 Dark Smoke Headlight ...	E	0.597935	-1.883189

```
{'candidate_count': 40, 'reranked_count': 5, 'query_encoding_ms': 26.183, 'semantic_scoring_ms': 0.154, 'rerank_ms': 169.112
```

```
from pandas.testing import assert_frame_equal

test_query_id = QUERY_IDS[0]
test_frame = evaluation_sample.iloc[query_rows[test_query_id]].reset_index(drop=True)
test_query = query_text[test_query_id]
n_test = len(test_frame)


def expect_value_error(*args, **kwargs):
    try:
        search_candidates(*args, **kwargs)
    except ValueError:
        return
    raise AssertionError(f"ValueError was expected for args={args[:1]} kwargs={list(kwargs)}")


def test_semantic_only_contract():
    results, stats = search_candidates(test_query, test_frame, top_n=5)
    assert len(results) == min(5, n_test)
    assert list(results["final_rank"]) == list(range(1, len(results) + 1))
    assert set(RESERVED_OUTPUT_COLUMNS) <= set(results.columns)
    assert np.all(np.diff(results["semantic_score"].to_numpy()) <= 0)
    assert results["cross_score"].isna().all()
    assert stats["candidate_count"] == n_test and stats["reranked_count"] == 0
    assert stats["rerank_ms"] == 0.0 or stats["rerank_ms"] < 1.0
    assert min(stats["query_encoding_ms"], stats["semantic_scoring_ms"]) >= 0


def test_selective_rerank_contract():
    k = min(5, n_test - 1)
    semantic_only, _ = search_candidates(test_query, test_frame, top_n=n_test)
    results, stats = search_candidates(test_query, test_frame, top_n=n_test, rerank_k=k)
    assert stats["reranked_count"] == k
    head, tail = results.iloc[:k], results.iloc[k:]
    assert head["cross_score"].notna().all() and tail["cross_score"].isna().all()
    assert np.all(np.diff(head["cross_score"].to_numpy()) <= 0)           # head sorted by cross score
    assert np.all(np.diff(tail["semantic_score"].to_numpy()) <= 0)        # tail keeps semantic order
    assert set(head["product_id"]) == set(semantic_only.iloc[:k]["product_id"])
    assert list(tail["product_id"]) == list(semantic_only.iloc[k:]["product_id"])


def test_size_limits():
    results, _ = search_candidates(test_query, test_frame, top_n=10 * n_test)
    assert len(results) == n_test
    _, stats = search_candidates(test_query, test_frame, top_n=3, rerank_k=10 * n_test)
    assert stats["reranked_count"] == n_test


def test_empty_candidates():
```

```
    results, stats = search_candidates(test_query, test_frame.iloc[0:0])
    assert results.empty and set(RESERVED_OUTPUT_COLUMNS) <= set(results.columns)
    assert stats["candidate_count"] == 0 and stats["reranked_count"] == 0


def test_input_is_not_modified():
    before = test_frame.copy()
    search_candidates(test_query, test_frame, top_n=5, rerank_k=5)
    assert_frame_equal(before, test_frame)


def test_invalid_arguments():
    for bad_query in ("", "   ", None, 123):
        expect_value_error(bad_query, test_frame)
    for bad_top_n in (0, -1, 1.5, True):
        expect_value_error(test_query, test_frame, top_n=bad_top_n)
    for bad_rerank_k in (-1, 2.5):
        expect_value_error(test_query, test_frame, rerank_k=bad_rerank_k)
    expect_value_error(test_query, test_frame.drop(columns="product_title"))
    expect_value_error(test_query, test_frame.assign(final_rank=0))


def test_title_handling_and_index_immutability():
    index_size, matrix_shape = len(title_to_idx), product_embeddings.shape
    expect_value_error(test_query, test_frame.assign(product_title=np.where(
        np.arange(n_test) == 0, None, test_frame["product_title"])))
    expect_value_error(test_query, test_frame.assign(product_title=np.where(
        np.arange(n_test) == 0, "  ", test_frame["product_title"])))
    expect_value_error(test_query, test_frame.assign(product_title=np.where(
        np.arange(n_test) == 0, "a title that is not in the index", test_frame["product_title"])))
    assert len(title_to_idx) == index_size and product_embeddings.shape == matrix_shape


def test_service_matches_evaluation_orderings():
    # The service and the Section 3 orderings use the same policy; compare on five queries.
    for query_id in QUERY_IDS[:5]:
        rows = query_rows[query_id]
        frame = evaluation_sample.iloc[rows].reset_index(drop=True)
        for method, k in ((SEMANTIC_NAME, 0), (selective_name(5), 5)):
            results, _ = search_candidates(query_text[query_id], frame, top_n=len(frame), rerank_k=k)
            expected = frame.iloc[orderings[method][query_id]]["product_id"].tolist()
            assert results["product_id"].tolist() == expected, (query_id, method)


service_tests = [
    test_semantic_only_contract, test_selective_rerank_contract, test_size_limits,
    test_empty_candidates, test_input_is_not_modified, test_invalid_arguments,
    test_title_handling_and_index_immutability, test_service_matches_evaluation_orderings,
]
for service_test in service_tests:
    service_test()          # any failure raises and stops the notebook
    print("PASS", service_test.__name__)
print(f"{len(service_tests)} service tests passed.")
```

```
PASS test_semantic_only_contract
PASS test_selective_rerank_contract
PASS test_size_limits
PASS test_empty_candidates
PASS test_input_is_not_modified
PASS test_invalid_arguments
PASS test_title_handling_and_index_immutability
PASS test_service_matches_evaluation_orderings
8 service tests passed.
```

```
# Operational benchmark: one query per call, as the service would receive it.
OPERATIONAL_MODES = [(SEMANTIC_NAME, 0), (selective_name(5), 5), (selective_name(20), 20)]
benchmark_inputs = [
    (query_text[query_id], evaluation_sample.iloc[query_rows[query_id]].reset_index(drop=True))
    for query_id in QUERY_IDS
]

search_candidates(*benchmark_inputs[0], top_n=10, rerank_k=5)   # warm-up, not recorded

call_records = []
for mode, k in OPERATIONAL_MODES:
    for query, frame in benchmark_inputs:
        start = time.perf_counter()
        _, stats = search_candidates(query, frame, top_n=10, rerank_k=k)
        stats["total_ms"] = (time.perf_counter() - start) * 1000
        call_records.append({"Mode": mode, **stats})

calls = pd.DataFrame(call_records)

```